

Android 動畫



// 透明漸變動畫

```
val anim = AlphaAnimation(1f, 0f)
anim.duration = 2000
anim.repeatCount = Animation.INFINITE

carImageView.startAnimation(anim)

// AlphaAnimation(1f, 0f) 用透明漸變動畫, 由不透明到完全透明
// duration 動畫持續時間
// repeatCount 動畫重覆次數, infinite 表示無限循環
```

// 位移動畫

```
val anim = TranslateAnimation(
    0f, 300f, 0f, 0f
)
anim.duration = 1000
anim.fillAfter = true
carImageView.startAnimation(anim)

// 300f 向 x 軸方向右位移 300f 作為結束位置
// fillAfter=true 動畫結束時保持結束時的位置, 不回到原點
```

// 縮放動畫

```
val anim = ScaleAnimation(1f, 2f, 1f, 2f)
anim.duration = 1000
anim.fillAfter = true
carImageView.startAnimation(anim)
```

// ScaleAnimation(1f, 2f, 1f, 2f) 分別表示 x 起始比例, x 結束比例, y 起始比例, y 結束比例
由原來的1倍放大到2倍

// 旋轉動畫

```
val anim = RotateAnimation(0f, -720f,
    Animation.RELATIVE_TO_SELF, 0.5f,
    Animation.RELATIVE_TO_SELF, 0.5f)
```

```
anim.duration = 2000
anim.repeatCount = Animation.INFINITE
carImageView.startAnimation(anim)
```

// -720 表示向逆時針方向旋轉兩次
// 0.5f 0.5f 控制旋轉中心點的 x, y 座標

// 同時套用兩種動畫

```
val animSet = AnimationSet(true)
animSet.addAnimation(anim)
animSet.addAnimation(anim2)

animSet.fillAfter = true

carImageView.startAnimation(animSet)
```

為動畫設定監聽器

```
val anim = RotateAnimation(0f, -360f,
    Animation.RELATIVE_TO_SELF, 0.5f,
    Animation.RELATIVE_TO_SELF, 0.5f)

anim.duration = 3000

anim.setAnimationListener(object: Animation.AnimationListener
{
    override fun onAnimationStart(animation: Animation?) {
        println("動畫開始時運行")
    }

    override fun onAnimationEnd(animation: Animation?) {
        println("動畫結束時運行")
    }

    override fun onAnimationRepeat(animation: Animation?) {
        println("動畫重新運行時運行")
    }
})

carImageView.startAnimation(anim)
```

Value Animator 動畫

```
val animator = ValueAnimator.ofInt(0, 100) // 在值0到100間進行動
畫
animator.duration = 2000
animator.interpolator = DecelerateInterpolator()

var progress = 0

// 動畫值變化期間更新 progressBar 的 progress 值

animator.addUpdateListener {
    progress = it.animatedValue as Int
    progressBar.progress = progress
}

animator.start()
```

使用 value Animator 不斷增加元素的寬度

```
val animator = ValueAnimator.ofInt(0, 700)
animator.duration = 1500
animator.interpolator = DecelerateInterpolator()

animator.addUpdateListener {
    val newWidth = it.animatedValue as Int
    your_view.layoutParams.width = newWidth
    your_view.requestLayout() // 重新計算佈局
}

animator.start()
```

Value Animator 動畫監聽器

```
animator.addListener(object : AnimatorListenerAdapter()
{
    override fun onAnimationEnd(animation: Animator) {
        super.onAnimationEnd(animation)
    }
}))
```